# CSA4704 - DEEP LEARNING
## Lab Experiments

**Name: Harini. R**
**Register no: 192324108**

**Experiment 21: Factor Analysis and Latent Factor Interpretation**
**Problem Statement: An HR department runs a 10-question employee-satisfaction survey. Management wants to know which few hidden factors drive the answers. Survey responses from 300 employees on pay, workload, manager support, growth, team spirit and similar items (Likert 1–5). The objective is to identify and interpret latent factors with factor analysis. Use Scikit-learn, NumPy, Pandas, Matplotlib.**
**Tasks:**
**1. Load and explore the survey data in Pandas; check correlations.**
**2. Fit Scikit-learn's FactorAnalysis with 2, 3 and 4 factors.**
**3. Apply varimax rotation and display the loadings matrix as a heatmap.**
**4. Name each factor from its high-loading questions (for example, "Compensation", "Work Culture").**
**5. Compare factor analysis with PCA on the same data.**

**Formulas:**

1. Factor analysis model

$$X = \Lambda F + \epsilon$$

Where:

- $X$ = observed variables
- $\Lambda$ = factor loading matrix
- $F$ = latent factors
- $\epsilon$ = unique error

2. Covariance matrix

$$\Sigma = \Lambda\Lambda^T + \Psi$$

Where $\Psi$ represents unique variances.

3. Variance explained by a factor

$$Varianceexplained = \frac{Factorvariance}{Totalvariance} \times 100$$

4. Varimax rotation objective

$$V = \sum_j \left[ \frac{1}{p} \sum_i \lambda_{ij}^4 - \left( \frac{1}{p} \sum_i \lambda_{ij}^2 \right)^2 \right]$$

Where $\lambda_{ij}$ is the loading of variable $i$ on factor $j$, and $p$ is the number of observed variables.

**Code:**

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.decomposition import FactorAnalysis, PCA
from sklearn.preprocessing import StandardScaler

np.random.seed(42)

# 1. Generate synthetic satisfaction survey data for 300 employees
n = 300

latent = np.random.normal(size=(n, 3))

questions = [
    "Pay", "Benefits", "Workload", "WorkLifeBalance",
    "ManagerSupport", "Communication", "TeamSpirit",
    "Growth", "Training", "Recognition"
]

# Three underlying factors: compensation, work culture, development
loadings_true = np.array([
    [0.9, 0.1, 0.1],
    [0.85, 0.1, 0.1],
    [0.1, -0.75, 0.1],
    [0.1, 0.8, 0.1],
    [0.1, 0.85, 0.1],
    [0.1, 0.75, 0.2],
    [0.1, 0.8, 0.1],
    [0.1, 0.1, 0.85],
    [0.1, 0.1, 0.8],
```

```
    [0.4, 0.2, 0.65]
])

continuous = latent @ loadings_true.T + np.random.normal(
    0, 0.6, size=(n, len(questions))
)

# Convert continuous scores into Likert responses from 1 to 5
survey = np.clip(np.rint(3 + continuous), 1, 5).astype(int)
df = pd.DataFrame(survey, columns=questions)

# 2. Explore the data
print("First five responses:")
display(df.head())

print("\nDescriptive statistics:")
display(df.describe().round(2))

print("\nCorrelation matrix:")
display(df.corr().round(2))

plt.figure(figsize=(10, 7))
sns.heatmap(df.corr(), annot=True, cmap="coolwarm", center=0)
plt.title("Employee Satisfaction Correlations")
plt.show()

# Standardize the survey responses
X = StandardScaler().fit_transform(df)

# 3. Fit Factor Analysis with 2, 3 and 4 factors
models = {}
for k in [2, 3, 4]:
    fa = FactorAnalysis(n_components=k, random_state=42)
    fa.fit(X)
    models[k] = fa
    print(f"{k} factors: log-likelihood = {fa.score(X) * len(X):.2f}")

# 4. Varimax rotation
def varimax(Phi, gamma=1.0, q=100, tol=1e-6):
    p, k = Phi.shape
    R = np.eye(k)
    old_objective = 0

    for _ in range(q):
```

```
        Lambda = Phi @ R
        u, s, vh = np.linalg.svd(
            Phi.T @ (
                Lambda**3
                - (gamma / p) * Lambda @ np.diag(np.diag(Lambda.T @ Lambda))
            )
        )
        R = u @ vh
        objective = np.sum(s)

        if old_objective != 0 and objective - old_objective < tol:
            break
        old_objective = objective

    return Phi @ R

# FactorAnalysis.components_ has shape (factors, questions)
# Transpose to get (questions, factors), then rotate.
for k, fa in models.items():
    loadings = fa.components_.T
    rotated = varimax(loadings)

    loadings_df = pd.DataFrame(
        rotated,
        index=questions,
        columns=[f"Factor {i+1}" for i in range(k)]
    )

    print(f"\nRotated loadings for {k} factors:")
    display(loadings_df.round(3))

    plt.figure(figsize=(8, 6))
    sns.heatmap(loadings_df, annot=True, cmap="coolwarm", center=0,
                vmin=-1, vmax=1)
    plt.title(f"Varimax-Rotated Factor Loadings ({k} Factors)")
    plt.show()

# 5. Interpret the three-factor solution
fa3 = models[3]
rotated3 = varimax(fa3.components_.T)

loadings3 = pd.DataFrame(
    rotated3,
    index=questions,
```

```
    columns=["Factor 1", "Factor 2", "Factor 3"]
)

print("\nQuestions with the strongest loading for each factor:")
for factor in loadings3.columns:
    print(f"\n{factor}:")
    print(loadings3[factor].abs().sort_values(ascending=False).head(4))

# Suggested interpretations based on the strongest-loading questions:
print("\nSuggested factor names:")
print("Compensation: Pay, Benefits")
print("Work Culture: Workload, WorkLifeBalance, ManagerSupport, "
      "Communication, TeamSpirit")
print("Growth and Recognition: Growth, Training, Recognition")

# 6. Compare Factor Analysis with PCA
pca = PCA(n_components=3)
pca.fit(X)

comparison = pd.DataFrame({
    "Component": ["1", "2", "3"],
    "PCA Explained Variance (%)":
        pca.explained_variance_ratio_ * 100
})

print("\nPCA explained variance:")
display(comparison.round(2))

print("Factor Analysis noise variance (per question):")
display(pd.Series(fa3.noise_variance_, index=questions, name="Noise Variance"))
```

## Output:

First five responses:

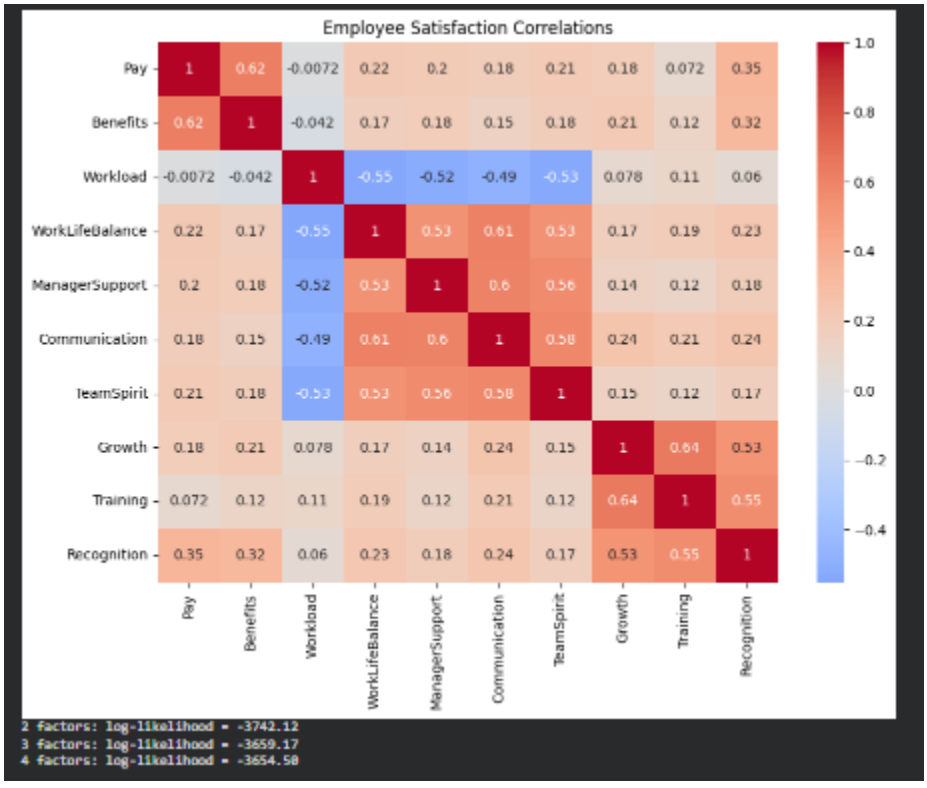
| | Pay | Benefits | Workload | WorkLifeBalance | ManagerSupport | Communication | TeamSpirit | Growth | Training | Recognition |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 4 | 3 | 3 | 4 | 3 | 3 | 2 | 4 | 4 | 5 |
| 1 | 4 | 4 | 3 | 4 | 3 | 3 | 3 | 3 | 3 | 4 |
| 2 | 5 | 4 | 3 | 3 | 4 | 3 | 3 | 3 | 4 | 3 |
| 3 | 3 | 4 | 2 | 1 | 3 | 2 | 2 | 3 | 3 | 2 |
| 4 | 2 | 3 | 5 | 1 | 2 | 1 | 1 | 1 | 1 | 2 |

Descriptive statistics:

| | Pay | Benefits | Workload | WorkLifeBalance | ManagerSupport | Communication | TeamSpirit | Growth | Training | Recognition |
|---|---|---|---|---|---|---|---|---|---|---|
| count | 300.00 | 300.00 | 300.00 | 300.00 | 300.00 | 300.00 | 300.00 | 300.00 | 300.00 | 300.00 |
| mean | 3.09 | 3.06 | 3.08 | 2.93 | 2.97 | 2.96 | 2.89 | 3.05 | 3.06 | 3.15 |
| std | 0.97 | 1.02 | 0.95 | 0.96 | 1.06 | 1.05 | 1.02 | 1.03 | 1.02 | 0.98 |
| min | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 |
| 25% | 2.00 | 2.00 | 3.00 | 2.00 | 2.00 | 2.00 | 2.00 | 2.00 | 2.00 | 3.00 |
| 50% | 3.00 | 3.00 | 3.00 | 3.00 | 3.00 | 3.00 | 3.00 | 3.00 | 3.00 | 3.00 |
| 75% | 4.00 | 4.00 | 4.00 | 4.00 | 4.00 | 4.00 | 4.00 | 4.00 | 4.00 | 4.00 |
| max | 5.00 | 5.00 | 5.00 | 5.00 | 5.00 | 5.00 | 5.00 | 5.00 | 5.00 | 5.00 |

Correlation matrix:

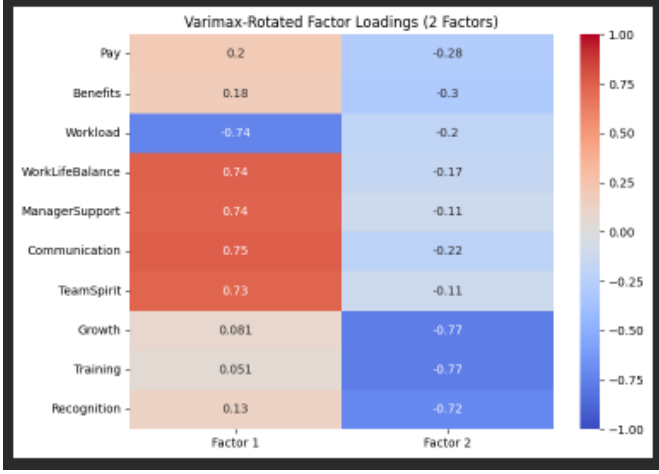
| | Pay | Benefits | Workload | WorkLifeBalance | ManagerSupport | Communication | TeamSpirit | Growth | Training | Recognition |
|---|---|---|---|---|---|---|---|---|---|---|
| Pay | 1.00 | 0.62 | -0.01 | 0.22 | 0.20 | 0.18 | 0.21 | 0.18 | 0.07 | 0.35 |
| Benefits | 0.62 | 1.00 | -0.04 | 0.17 | 0.18 | 0.15 | 0.18 | 0.21 | 0.12 | 0.32 |
| Workload | -0.01 | -0.04 | 1.00 | -0.55 | -0.52 | -0.49 | -0.53 | 0.08 | 0.11 | 0.06 |
| WorkLifeBalance | 0.22 | 0.17 | -0.55 | 1.00 | 0.53 | 0.61 | 0.53 | 0.17 | 0.19 | 0.23 |
| ManagerSupport | 0.20 | 0.18 | -0.52 | 0.53 | 1.00 | 0.60 | 0.56 | 0.14 | 0.12 | 0.18 |
| Communication | 0.18 | 0.15 | -0.49 | 0.61 | 0.60 | 1.00 | 0.58 | 0.24 | 0.21 | 0.24 |
| TeamSpirit | 0.21 | 0.18 | -0.53 | 0.53 | 0.56 | 0.58 | 1.00 | 0.15 | 0.12 | 0.17 |
| Growth | 0.18 | 0.21 | 0.08 | 0.17 | 0.14 | 0.24 | 0.15 | 1.00 | 0.64 | 0.53 |
| Training | 0.07 | 0.12 | 0.11 | 0.19 | 0.12 | 0.21 | 0.12 | 0.64 | 1.00 | 0.55 |
| Recognition | 0.35 | 0.32 | 0.06 | 0.23 | 0.18 | 0.24 | 0.17 | 0.53 | 0.55 | 1.00 |




```
2 factors: log-likelihood = -3742.12
3 factors: log-likelihood = -3659.17
4 factors: log-likelihood = -3654.50
```

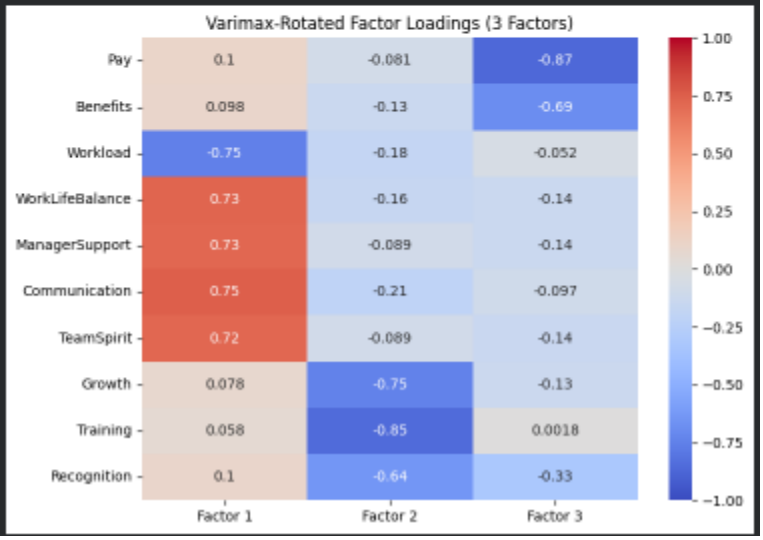
Rotated loadings for 2 factors:

| | Factor 1 | Factor 2 |
|---|---|---|
| Pay | 0.201 | -0.284 |
| Benefits | 0.178 | -0.297 |
| Workload | -0.739 | -0.201 |
| WorkLifeBalance | 0.743 | -0.170 |
| Manager Support | 0.738 | -0.109 |
| Communication | 0.754 | -0.219 |
| Team Spirit | 0.733 | -0.110 |
| Growth | 0.081 | -0.770 |
| Training | 0.051 | -0.767 |
| Recognition | 0.131 | -0.716 |



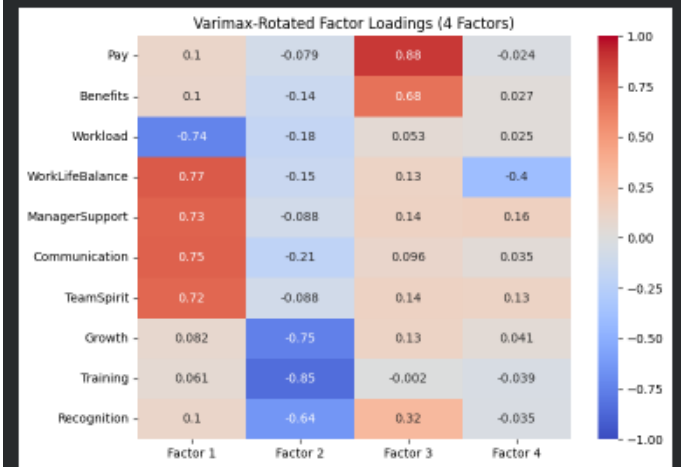

Rotated loadings for 3 factors:

| | Factor 1 | Factor 2 | Factor 3 |
|---|---|---|---|
| Pay | 0.102 | -0.081 | -0.872 |
| Benefits | 0.098 | -0.134 | -0.688 |
| Workload | -0.752 | -0.177 | -0.052 |
| WorkLifeBalance | 0.734 | -0.157 | -0.135 |
| Manager Support | 0.726 | -0.089 | -0.137 |
| Communication | 0.752 | -0.209 | -0.097 |
| Team Spirit | 0.721 | -0.089 | -0.141 |
| Growth | 0.078 | -0.748 | -0.133 |
| Training | 0.058 | -0.853 | 0.002 |
| Recognition | 0.101 | -0.642 | -0.327 |



Rotated loadings for 4 factors:

| | Factor 1 | Factor 2 | Factor 3 | Factor 4 |
|---|---|---|---|---|
| Pay | 0.103 | -0.079 | 0.885 | -0.024 |
| Benefits | 0.101 | -0.137 | 0.679 | 0.027 |
| Workload | -0.741 | -0.179 | 0.053 | 0.025 |
| WorkLifeBalance | 0.773 | -0.148 | 0.128 | -0.401 |
| ManagerSupport | 0.734 | -0.088 | 0.135 | 0.156 |
| Communication | 0.749 | -0.206 | 0.096 | 0.035 |
| TeamSpirit | 0.723 | -0.088 | 0.140 | 0.126 |
| Growth | 0.082 | -0.752 | 0.131 | 0.041 |
| Training | 0.061 | -0.850 | -0.002 | -0.039 |
| Recognition | 0.104 | -0.641 | 0.324 | -0.035 |




Questions with the strongest loading for each factor:

```
Factor 1:
Communication      0.751793
Workload           0.751577
WorkLifeBalance    0.734286
ManagerSupport     0.726295
Name: Factor 1, dtype: float64

Factor 2:
Training           0.852686
Growth             0.748363
Recognition        0.641973
Communication      0.289105
Name: Factor 2, dtype: float64

Factor 3:
Pay              0.871914
Benefits         0.687632
Recognition      0.327345
TeamSpirit       0.141449
Name: Factor 3, dtype: float64

Suggested factor names:
Compensation: Pay, Benefits
Work Culture: Workload, WorkLifeBalance, ManagerSupport, Communication, TeamSpirit
Growth and Recognition: Growth, Training, Recognition

PCA explained variance:
```

| | Component | PCA Explained Variance (%) |
|---|---|---|
| 0 | 1 | 36.42 |
| 1 | 2 | 21.54 |
| 2 | 3 | 13.60 |

Factor Analysis noise variance (per question):

| | Noise Variance |
|---|---|
| Pay | 0.224414 |
| Benefits | 0.498676 |
| Workload | 0.401300 |
| WorkLifeBalance | 0.417817 |
| Manager Support | 0.445827 |
| Communication | 0.381580 |
| Team Spirit | 0.451456 |
| Growth | 0.416193 |
| Training | 0.269470 |
| Recognition | 0.470421 |

dtype: float64

**Experiment 22: Independent Component Analysis and Component Visualization**
**Problem Statement: In a hospital ECG recording, the mother's heartbeat masks the foetal heartbeat. Doctors need the two signals separated (the cocktail-party problem). Three synthetic source signals (sine, square and sawtooth, or maternal and foetal ECG) mixed through a random mixing matrix. The objective is to separate mixed signals into independent sources with ICA. Use Scikit-learn, NumPy, Matplotlib.**
**Tasks:**
**1. Generate the source signals and mix them.**
**2. Apply FastICA to recover the sources.**
**3. Apply PCA to the same mixtures for comparison.**
**4. Plot original, mixed, ICA-recovered and PCA-recovered signals.**
**5. Measure recovery quality with correlation to the true sources.**

**Formulas:**

1. Mixing model

$$X = AS$$

Where:

- $X$ = observed mixed signals
- $A$ = mixing matrix
- $S$ = original independent sources

2. Source separation

$$\hat{S} = WX$$

Where $W$ is the unmixing matrix.

3. Whitening transformation

$$Z = D^{-1/2}E^{T}(X - \mu)$$

Where $E$ contains covariance eigenvectors and $D$ contains eigenvalues.

4. ICA independence objective

$$\max_{W} \sum_{i} H(\hat{S}_i)$$

Here $H$ represents entropy. ICA seeks statistically independent source signals; practical algorithms often use approximations to this objective.

**Code:**

```
import numpy as np
import matplotlib.pyplot as plt

from scipy.signal import sawtooth, square
from sklearn.decomposition import FastICA, PCA
from sklearn.preprocessing import StandardScaler

np.random.seed(42)

# 1. Generate three independent source signals
n_samples = 2000
t = np.linspace(0, 8, n_samples)

S = np.c_[
    np.sin(2 * t),              # Sine wave
    square(2 * np.pi * 0.5 * t),   # Square wave
    sawtooth(2 * np.pi * 0.5 * t)  # Sawtooth wave
]

# Center and scale the sources
S = StandardScaler().fit_transform(S)

# 2. Mix the source signals using a random mixing matrix
A = np.array([
    [1.0, 0.5, 0.2],
```

```
    [0.4, 1.0, 0.6],
    [0.3, 0.2, 1.0]
])

X = S @ A.T

print("Mixing matrix:\n", A)
print("Mixed signal shape:", X.shape)

# 3. Recover sources using FastICA
ica = FastICA(n_components=3, random_state=42, max_iter=2000,
              tol=0.001, whiten="unit-variance")
S_ica = ica.fit_transform(X)

# 4. Apply PCA to the same mixtures
pca = PCA(n_components=3)
S_pca = pca.fit_transform(X)

# Plot original sources
fig, axes = plt.subplots(3, 1, figsize=(12, 7))
for i in range(3):
    axes[i].plot(t[:500], S[:500, i])
    axes[i].set_title(f"Original Source {i+1}")
plt.tight_layout()
plt.show()

# Plot mixed signals
fig, axes = plt.subplots(3, 1, figsize=(12, 7))
for i in range(3):
    axes[i].plot(t[:500], X[:500, i])
    axes[i].set_title(f"Mixed Signal {i+1}")
plt.tight_layout()
plt.show()

# Plot ICA recovered signals
fig, axes = plt.subplots(3, 1, figsize=(12, 7))
for i in range(3):
    axes[i].plot(t[:500], S_ica[:500, i])
    axes[i].set_title(f"ICA Recovered Component {i+1}")
plt.tight_layout()
plt.show()

# Plot PCA components
fig, axes = plt.subplots(3, 1, figsize=(12, 7))
```

```
for i in range(3):
    axes[i].plot(t[:500], S_pca[:500, i])
    axes[i].set_title(f"PCA Component {i+1}")
plt.tight_layout()
plt.show()

# 5. Measure recovery quality using absolute correlation.
# ICA components may appear in a different order and sign.
def best_correlations(reference, recovered):
    corr = np.corrcoef(reference.T, recovered.T)[:3, 3:]
    best = np.max(np.abs(corr), axis=1)
    return best, corr

ica_best, ica_corr = best_correlations(S, S_ica)
pca_best, pca_corr = best_correlations(S, S_pca)

print("Best absolute correlation for each original source:")
print("ICA:", np.round(ica_best, 3))
print("PCA:", np.round(pca_best, 3))

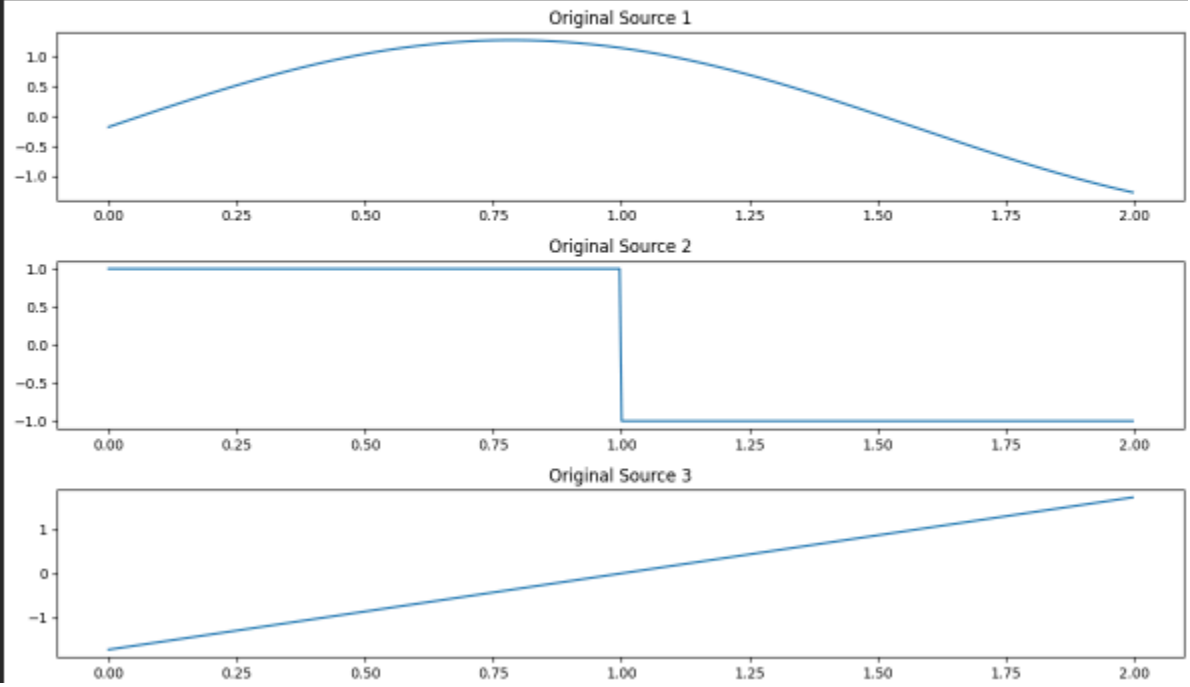
print("\nFull absolute correlation matrix: original vs ICA")
print(np.round(np.abs(ica_corr), 3))

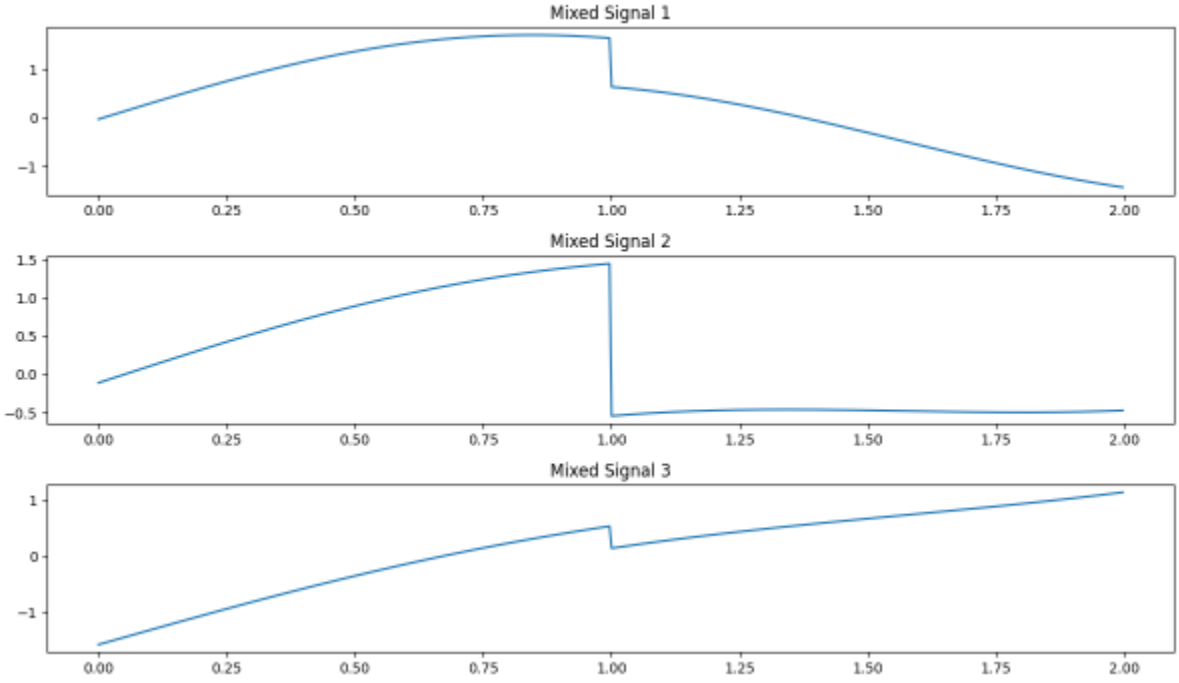
print("\nFull absolute correlation matrix: original vs PCA")
print(np.round(np.abs(pca_corr), 3))

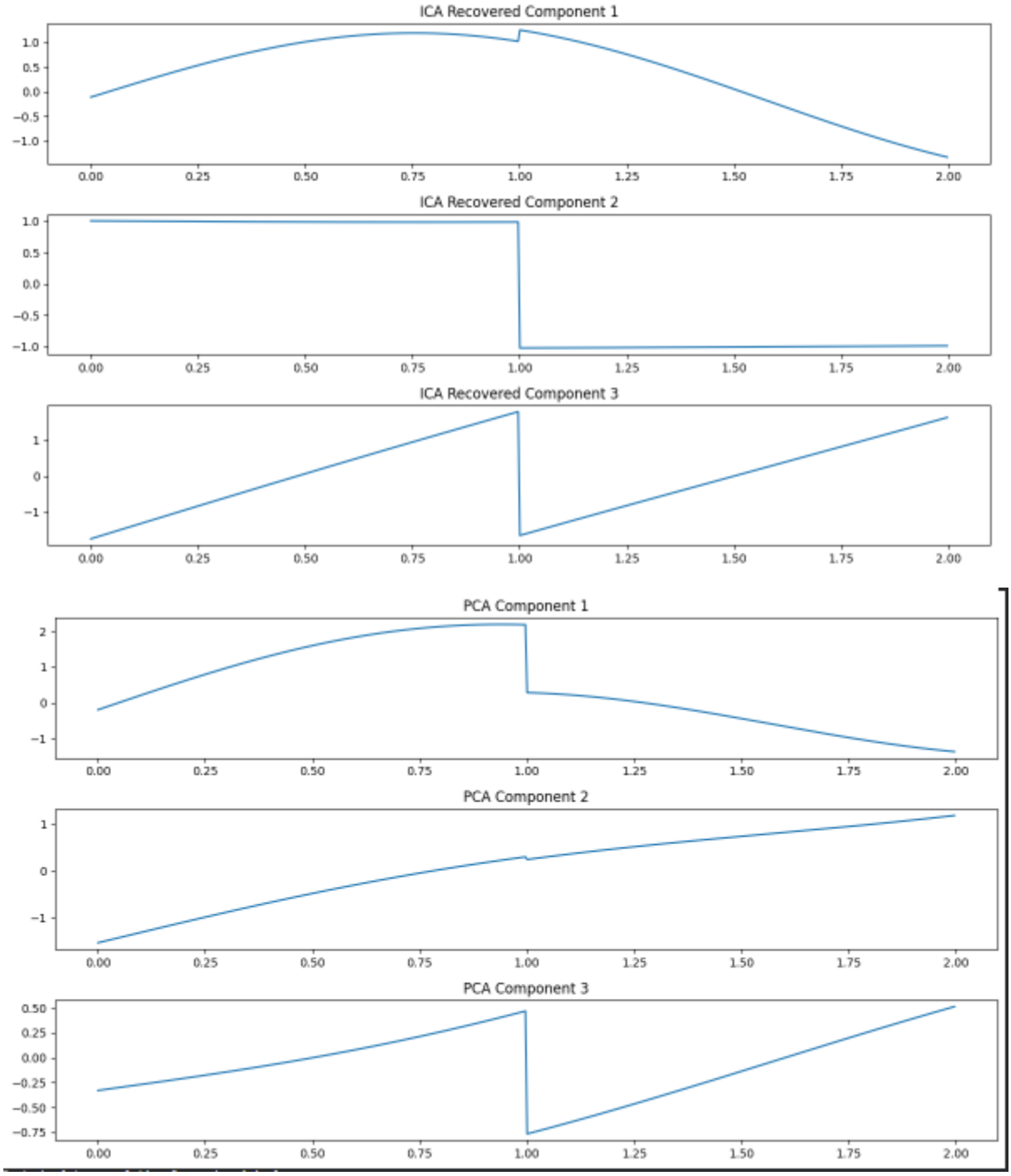
plt.figure(figsize=(8, 4))
plt.bar(
    ["Source 1", "Source 2", "Source 3"],
    ica_best, label="FastICA", alpha=0.8
)
plt.bar(
    ["Source 1", "Source 2", "Source 3"],
    pca_best, label="PCA", alpha=0.8
)
plt.ylabel("Best absolute correlation")
plt.title("Signal Recovery Quality")
plt.ylim(0, 1.05)
plt.legend()
plt.show()
```

**Output:**

```
Mixing matrix:
 [[1.  0.5 0.2]
 [0.4 1.  0.6]
 [0.3 0.2 1. ]]
Mixed signal shape: (2000, 3)
```



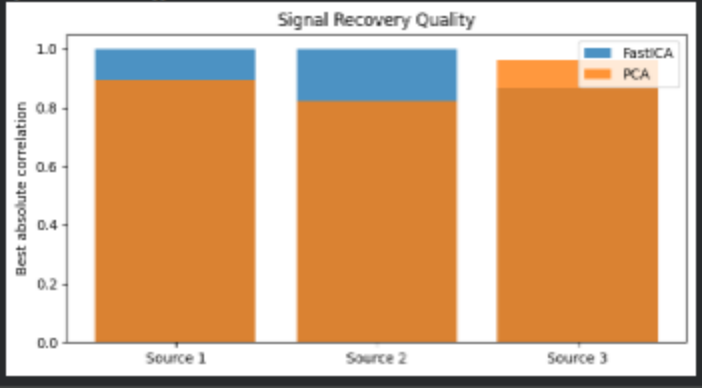




ICA Recovered Component 1
ICA Recovered Component 2
ICA Recovered Component 3
PCA Component 1
PCA Component 2
PCA Component 3

```
Best absolute correlation for each original source:
ICA: [0.998 1.    0.866]
PCA: [0.895 0.823 0.962]

Full absolute correlation matrix: original vs ICA
[[0.998 0.026 0.053]
 [0.014 1.    0.001]
 [0.046 0.866 0.498]]

Full absolute correlation matrix: original vs PCA
[[0.895 0.233 0.381]
 [0.421 0.823 0.381]
 [0.259 0.962 0.091]]
```




**Experiment 23: Bayesian Network Construction and Visualization**
**Problem Statement: A clinic wants a decision-support tool that reasons about the causes of a patient's symptoms under uncertainty. A medical diagnosis network with nodes Smoking, Pollution, Lung Disease, Cough, X-ray Result and Fatigue, with given conditional probability tables. The objective is to build a Bayesian network, define its CPDs and visualise its structure. Use pgmpy, NetworkX, Matplotlib.**
**Tasks:**
**1. Define the directed acyclic graph in pgmpy.**
**2. Specify the conditional probability distribution for every node.**
**3. Validate the model with check_model().**
**4. Draw the network with NetworkX.**
**5. List the conditional independencies and active trails in the network.**

**Formulas:**

1. Energy function

$$E(v,h) = -a^T v - b^T h - v^T W h$$

2. Hidden-unit probability

$$P(h_j = 1|v) = \sigma\left(b_j + \sum_i v_i W_{ij}\right)$$

3. Visible-unit probability

$$P(v_i = 1|h) = \sigma\left(a_i + \sum_j W_{ij} h_j\right)$$

4. Reconstruction error

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(v_i - \hat{v}_i)^2$$

**Code:**

```
import networkx as nx
import matplotlib.pyplot as plt

from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD

# 1. Define the directed acyclic graph
edges = [
   ("Smoking", "LungDisease"),
   ("Pollution", "LungDisease"),
   ("LungDisease", "Cough"),
   ("LungDisease", "Xray"),
   ("LungDisease", "Fatigue")
]

model = DiscreteBayesianNetwork(edges)

# States: 0 = No, 1 = Yes
# 2. Define the CPD for each node

cpd_smoking = TabularCPD(
   variable="Smoking", variable_card=2,
   values=[[0.7], [0.3]]
)

cpd_pollution = TabularCPD(
   variable="Pollution", variable_card=2,
   values=[[0.8], [0.2]]
)
```

```
# Parent order: Smoking, Pollution
# Columns: (0,0), (0,1), (1,0), (1,1)
cpd_disease = TabularCPD(
    variable="LungDisease", variable_card=2,
    values=[
        [0.99, 0.95, 0.90, 0.80],  # No disease
        [0.01, 0.05, 0.10, 0.20]   # Disease
    ],
    evidence=["Smoking", "Pollution"],
    evidence_card=[2, 2]
)

cpd_cough = TabularCPD(
    variable="Cough", variable_card=2,
    values=[
        [0.95, 0.20],  # No cough
        [0.05, 0.80]   # Cough
    ],
    evidence=["LungDisease"],
    evidence_card=[2]
)

cpd_xray = TabularCPD(
    variable="Xray", variable_card=2,
    values=[
        [0.98, 0.10],  # Negative X-ray
        [0.02, 0.90]   # Positive X-ray
    ],
    evidence=["LungDisease"],
    evidence_card=[2]
)

cpd_fatigue = TabularCPD(
    variable="Fatigue", variable_card=2,
    values=[
        [0.90, 0.30],  # No fatigue
        [0.10, 0.70]   # Fatigue
    ],
    evidence=["LungDisease"],
    evidence_card=[2]
)

model.add_cpds(
    cpd_smoking, cpd_pollution, cpd_disease,
```

```
    cpd_cough, cpd_xray, cpd_fatigue
)

# 3. Validate the Bayesian network
print("Is the model valid?", model.check_model())

print("\nConditional Probability Distributions:")
for cpd in model.get_cpds():
    print("\n", cpd)

# 4. Visualize the network
G = nx.DiGraph()
G.add_edges_from(edges)

pos = {
    "Smoking": (0, 2),
    "Pollution": (2, 2),
    "LungDisease": (1, 1),
    "Cough": (0, 0),
    "Xray": (1, 0),
    "Fatigue": (2, 0)
}

plt.figure(figsize=(9, 6))
nx.draw(
    G, pos, with_labels=True, node_size=3000,
    node_color="lightblue", font_size=9,
    arrows=True, arrowsize=20
)
plt.title("Medical Bayesian Network")
plt.show()

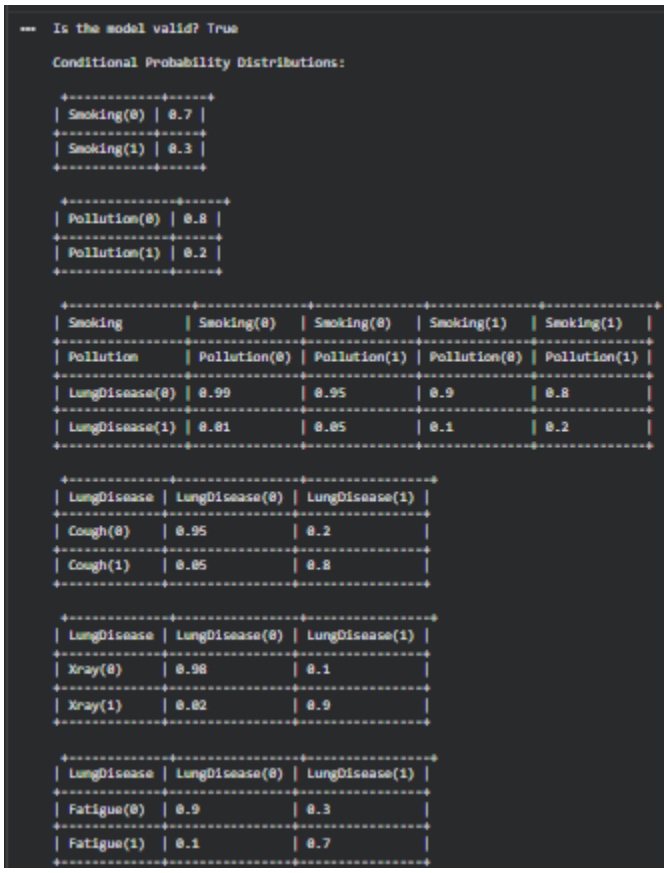
# 5. Explore conditional independencies
print("\nConditional independencies reported by the model:")
try:
    print(model.get_independencies())
except Exception as e:
    print("Could not list all independencies automatically:", e)

# Active trails from Smoking with no evidence
try:
    print("\nActive trails from Smoking, no evidence:")
    print(model.active_trail_nodes("Smoking"))
except Exception as e:
```

```
    print("Active-trail API differs in this pgmpy version:", e)

# Active trails after observing LungDisease
try:
    print("\nActive trails from Smoking, observing LungDisease:")
    print(model.active_trail_nodes("Smoking", observed=["LungDisease"]))
except Exception as e:
    print("Active-trail API differs in this pgmpy version:", e)
```

**Output:**



```
=== Is the model valid? True

    Conditional Probability Distributions:

     +------------+-----+
    | Smoking(0) | 0.7 |
    +------------+-----+
    | Smoking(1) | 0.3 |
    +------------+-----+

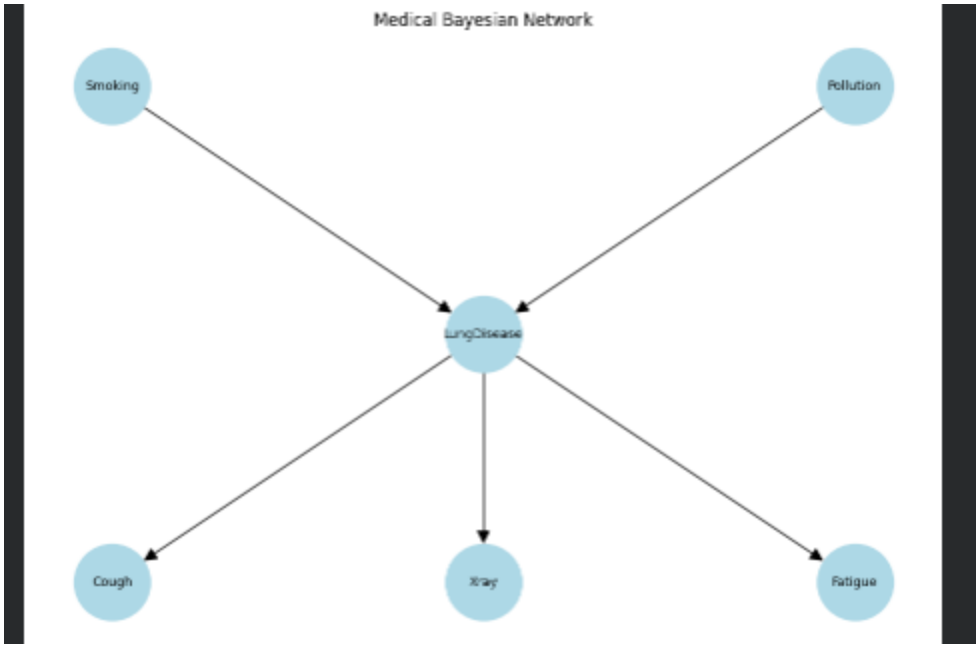
     +--------------+-----+
    | Pollution(0) | 0.8 |
    +--------------+-----+
    | Pollution(1) | 0.2 |
    +--------------+-----+

     +---------------+--------------+--------------+--------------+--------------+
    | Smoking       | Smoking(0)   | Smoking(0)   | Smoking(1)   | Smoking(1)   |
    +---------------+--------------+--------------+--------------+--------------+
    | Pollution     | Pollution(0) | Pollution(1) | Pollution(0) | Pollution(1) |
    +---------------+--------------+--------------+--------------+--------------+
    | LungDisease(0) | 0.99        | 0.95         | 0.9          | 0.8          |
    +---------------+--------------+--------------+--------------+--------------+
    | LungDisease(1) | 0.01        | 0.05         | 0.1          | 0.2          |
    +---------------+--------------+--------------+--------------+--------------+

     +-------------+----------------+----------------+
    | LungDisease | LungDisease(0) | LungDisease(1) |
    +-------------+----------------+----------------+
    | Cough(0)    | 0.95           | 0.2            |
    +-------------+----------------+----------------+
    | Cough(1)    | 0.05           | 0.8            |
    +-------------+----------------+----------------+

     +-------------+----------------+----------------+
    | LungDisease | LungDisease(0) | LungDisease(1) |
    +-------------+----------------+----------------+
    | Xray(0)     | 0.98           | 0.1            |
    +-------------+----------------+----------------+
    | Xray(1)     | 0.02           | 0.9            |
    +-------------+----------------+----------------+

     +-------------+----------------+----------------+
    | LungDisease | LungDisease(0) | LungDisease(1) |
    +-------------+----------------+----------------+
    | Fatigue(0)  | 0.9            | 0.3            |
    +-------------+----------------+----------------+
    | Fatigue(1)  | 0.1            | 0.7            |
    +-------------+----------------+----------------+
```




```
Conditional independencies reported by the model:
(Fatigue ⊥ Pollution | LungDisease)
(Cough ⊥ Xray | LungDisease)
(Fatigue ⊥ Xray | LungDisease)
(Cough ⊥ Pollution | LungDisease)
(Fatigue ⊥ Smoking | LungDisease)
(Smoking ⊥ Xray | LungDisease)
(Pollution ⊥ Smoking)
(Cough ⊥ Smoking | LungDisease)
(Pollution ⊥ Xray | LungDisease)
(Cough ⊥ Fatigue | LungDisease)

Active trails from Smoking, no evidence:
{'Smoking': {'Cough', 'Fatigue', 'LungDisease', 'Xray', 'Smoking'}}

Active trails from Smoking, observing LungDisease:
{'Smoking': {'Smoking', 'Pollution'}}
```

**Experiment 24: Graphical Model Sampling and Probabilistic Inference**

**Problem Statement: The clinic from Experiment 23 asks: given a positive X-ray and a persistent cough, how likely is lung disease, and does smoking change the answer? The same medical Bayesian network, or a student-performance network (Difficulty, Intelligence, Grade, SAT, Letter). The objective is to perform exact and approximate inference on a graphical model. Use pgmpy, NumPy, Pandas, NetworkX, Matplotlib.**

**Tasks:**

**1. Run exact inference with Variable Elimination for several evidence settings.**

**2. Generate samples with forward sampling and rejection sampling.**

**3. Estimate the same queries from the samples using Pandas.**

**4. Plot estimated probability against sample size to show convergence.**

**5. Compare exact and approximate answers in a table.**

**Formulas:**

1. RBM energy function

$$E(v,h) = -a^T v - b^T h - v^T W h$$

2. Hidden representation

$$h = \sigma(W^T v + b)$$

3. Layer-wise pretraining

$$h^{(1)} = \sigma(W_1^T x + b_1)$$

$$h^{(2)} = \sigma(W_2^T h^{(1)} + b_2)$$

4. Cross-entropy loss

$$L = -\sum_{i=1}^{C} y_i \log(\hat{y}_i)$$

**Code:**

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
from pgmpy.sampling import BayesianModelSampling

np.random.seed(42)

# 1. Rebuild the Bayesian network
edges = [
    ("Smoking", "LungDisease"),
    ("Pollution", "LungDisease"),
    ("LungDisease", "Cough"),
```

```
    ("LungDisease", "Xray"),
    ("LungDisease", "Fatigue")
]

model = DiscreteBayesianNetwork(edges)

cpds = [
    TabularCPD("Smoking", 2, [[0.7], [0.3]]),
    TabularCPD("Pollution", 2, [[0.8], [0.2]]),

    TabularCPD(
        "LungDisease", 2,
        [[0.99, 0.95, 0.90, 0.80],
         [0.01, 0.05, 0.10, 0.20]],
        evidence=["Smoking", "Pollution"],
        evidence_card=[2, 2]
    ),

    TabularCPD(
        "Cough", 2,
        [[0.95, 0.20],
         [0.05, 0.80]],
        evidence=["LungDisease"],
        evidence_card=[2]
    ),

    TabularCPD(
        "Xray", 2,
        [[0.98, 0.10],
         [0.02, 0.90]],
        evidence=["LungDisease"],
        evidence_card=[2]
    ),

    TabularCPD(
        "Fatigue", 2,
        [[0.90, 0.30],
         [0.10, 0.70]],
        evidence=["LungDisease"],
        evidence_card=[2]
    )
]

model.add_cpds(*cpds)
```

```
assert model.check_model()

# 2. Exact inference using Variable Elimination
infer = VariableElimination(model)

evidence_cases = [
    {"Xray": 1, "Cough": 1},
    {"Xray": 1, "Cough": 1, "Smoking": 1},
    {"Xray": 1, "Cough": 1, "Smoking": 0},
    {"Cough": 1},
    {"Xray": 1}
]

exact_results = []

for evidence in evidence_cases:
    query = infer.query(
        variables=["LungDisease"],
        evidence=evidence,
        show_progress=False
    )

    probability = float(query.values[1])
    exact_results.append({
        "Evidence": str(evidence),
        "P(Disease = Yes)": probability
    })

exact_df = pd.DataFrame(exact_results)
print("Exact inference results:")
display(exact_df.round(4))

# 3. Generate samples using forward sampling
sampler = BayesianModelSampling(model)

N = 100000
samples = sampler.forward_sample(
    size=N, seed=42, show_progress=False
)

print("\nFirst five forward samples:")
display(samples.head())

# Estimate conditional probability from samples
```

```
def estimate_from_samples(data, evidence):
    mask = np.ones(len(data), dtype=bool)

    for variable, state in evidence.items():
        mask &= (data[variable].to_numpy() == state)

    selected = data.loc[mask]
    if len(selected) == 0:
        return np.nan, 0

    return (
        float((selected["LungDisease"] == 1).mean()),
        len(selected)
    )

sampling_results = []

for evidence in evidence_cases:
    estimate, count = estimate_from_samples(samples, evidence)
    sampling_results.append({
        "Evidence": str(evidence),
        "Sample Estimate": estimate,
        "Matching Samples": count
    })

sampling_df = pd.DataFrame(sampling_results)
print("\nForward-sampling estimates:")
display(sampling_df.round(4))

# 4. Rejection sampling: generate samples and keep only those
# matching the evidence
def rejection_sampling(data, evidence):
    mask = np.ones(len(data), dtype=bool)
    for variable, state in evidence.items():
        mask &= (data[variable].to_numpy() == state)

    accepted = data.loc[mask]
    if len(accepted) == 0:
        return np.nan, 0

    return float((accepted["LungDisease"] == 1).mean()), len(accepted)

rejection_results = []
```

```
for evidence in evidence_cases:
    estimate, accepted_count = rejection_sampling(samples, evidence)
    rejection_results.append({
        "Evidence": str(evidence),
        "Rejection Estimate": estimate,
        "Accepted Samples": accepted_count
    })

rejection_df = pd.DataFrame(rejection_results)
print("\nRejection-sampling estimates:")
display(rejection_df.round(4))

# 5. Plot convergence for P(Disease | positive X-ray, cough)
evidence = {"Xray": 1, "Cough": 1}
sizes = [1000, 5000, 10000, 25000, 50000, 100000]

exact_query = infer.query(
    variables=["LungDisease"],
    evidence=evidence,
    show_progress=False
)
exact_probability = float(exact_query.values[1])

forward_estimates = []
rejection_estimates = []

for size in sizes:
    subset = samples.iloc[:size]

    forward_estimates.append(
        estimate_from_samples(subset, evidence)[0]
    )

    rejection_estimates.append(
        rejection_sampling(subset, evidence)[0]
    )

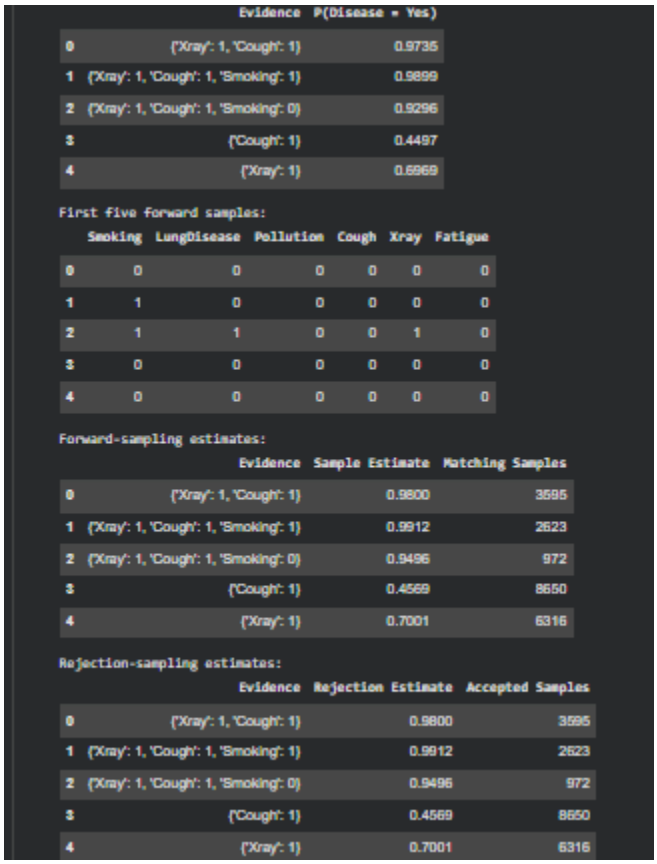
plt.figure(figsize=(9, 5))
plt.plot(sizes, forward_estimates, marker="o", label="Forward sampling")
plt.plot(sizes, rejection_estimates, marker="s", label="Rejection sampling")
plt.axhline(
    exact_probability, linestyle="--",
    label=f"Exact probability = {exact_probability:.3f}"
)
```

```
plt.xlabel("Number of Generated Samples")
plt.ylabel("Estimated P(LungDisease = Yes)")
plt.title("Approximate Inference Convergence")
plt.legend()
plt.grid(True)
plt.show()

# 6. Compare exact and approximate answers in one table
final_table = exact_df.merge(sampling_df, on="Evidence")
final_table = final_table.merge(rejection_df, on="Evidence")

display(final_table.round(4))
```

**Output:**



| | Evidence | P(Disease = Yes) |
|---|---|---|
| 0 | {'Xray': 1, 'Cough': 1} | 0.9735 |
| 1 | {'Xray': 1, 'Cough': 1, 'Smoking': 1} | 0.9899 |
| 2 | {'Xray': 1, 'Cough': 1, 'Smoking': 0} | 0.9296 |
| 3 | {'Cough': 1} | 0.4497 |
| 4 | {'Xray': 1} | 0.6969 |

First five forward samples:

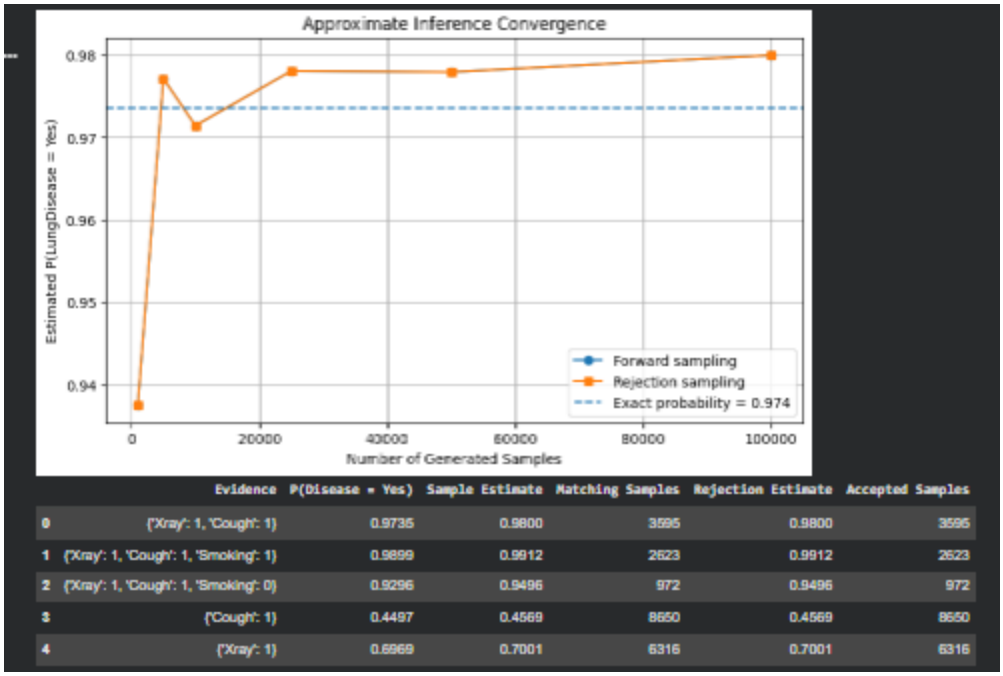
| | Smoking | LungDisease | Pollution | Cough | Xray | Fatigue |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 | 0 | 0 |
| 2 | 1 | 1 | 0 | 0 | 1 | 0 |
| 3 | 0 | 0 | 0 | 0 | 0 | 0 |
| 4 | 0 | 0 | 0 | 0 | 0 | 0 |

Forward-sampling estimates:

| | Evidence | Sample Estimate | Matching Samples |
|---|---|---|---|
| 0 | {'Xray': 1, 'Cough': 1} | 0.9800 | 3595 |
| 1 | {'Xray': 1, 'Cough': 1, 'Smoking': 1} | 0.9912 | 2623 |
| 2 | {'Xray': 1, 'Cough': 1, 'Smoking': 0} | 0.9496 | 972 |
| 3 | {'Cough': 1} | 0.4569 | 8650 |
| 4 | {'Xray': 1} | 0.7001 | 6316 |

Rejection-sampling estimates:

| | Evidence | Rejection Estimate | Accepted Samples |
|---|---|---|---|
| 0 | {'Xray': 1, 'Cough': 1} | 0.9800 | 3595 |
| 1 | {'Xray': 1, 'Cough': 1, 'Smoking': 1} | 0.9912 | 2623 |
| 2 | {'Xray': 1, 'Cough': 1, 'Smoking': 0} | 0.9496 | 972 |
| 3 | {'Cough': 1} | 0.4569 | 8650 |
| 4 | {'Xray': 1} | 0.7001 | 6316 |




| | Evidence | P(Disease = Yes) | Sample Estimate | Matching Samples | Rejection Estimate | Accepted Samples |
|---|---|---|---|---|---|---|
| 0 | {'Xray': 1, 'Cough': 1} | 0.9735 | 0.9800 | 3595 | 0.9800 | 3595 |
| 1 | {'Xray': 1, 'Cough': 1, 'Smoking': 1} | 0.9899 | 0.9912 | 2623 | 0.9912 | 2623 |
| 2 | {'Xray': 1, 'Cough': 1, 'Smoking': 0} | 0.9296 | 0.9496 | 972 | 0.9496 | 972 |
| 3 | {'Cough': 1} | 0.4497 | 0.4569 | 8650 | 0.4569 | 8650 |
| 4 | {'Xray': 1} | 0.6969 | 0.7001 | 6316 | 0.7001 | 6316 |

**CO5 – Energy-Based and Generative Models**

**Experiment 25: Restricted Boltzmann Machine Energy and Reconstruction**

**Problem Statement: A movie-streaming service wants to learn hidden taste patterns from users' binary watched/not-watched records and fill in likely preferences. A binary user–movie matrix (or binarised MNIST digits) with 784 visible units and 64 hidden units. The objective is to implement an RBM, compute its energy function and reconstruct inputs. Use NumPy, SciPy, Matplotlib.**

**Tasks:**

**1. Implement the energy function E(v, h) = −aTv − bTh − vTWh in NumPy.**

**2. Compute hidden activation probabilities p(h|v) and visible probabilities p(v|h).**

**3. Train the RBM with one-step contrastive divergence.**

**4. Plot average free energy and reconstruction error over epochs.**

**5. Show original and reconstructed samples side by side.**

**Formulas:**

1. Energy function for two hidden layers

$$E(v, h^{(1)}, h^{(2)}) = -\, v^T W_1 h^{(1)} - (h^{(1)})^T W_2 h^{(2)} - a^T v - b_1^T h^{(1)} - b_2^T h^{(2)}$$

2. Hidden-layer probability

$$P(h_i^{(1)} = 1 | v, h^{(2)}) = \sigma\left(b_i^{(1)} + \sum_j W_{1,ji} v_j + \sum_k W_{2,ik} h_k^{(2)}\right)$$

3. Visible-layer probability

$$P(v_i = 1|h^{(1)}) = \sigma\left(a_i + \sum_j W_{1,ij} h_j^{(1)}\right)$$

4. Reconstruction error

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(v_i - \hat{v}_i)^2$$

**Code:**

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist

np.random.seed(42)

# 1. Load and binarize MNIST
(x_train, _), (x_test, _) = mnist.load_data()

# Scale pixel values to [0, 1], then binarize
x_train = (x_train.reshape(-1, 784) / 255.0 > 0.5).astype(np.float32)
x_test = (x_test.reshape(-1, 784) / 255.0 > 0.5).astype(np.float32)

# Use subsets to keep the NumPy implementation manageable
x_train = x_train[:10000]
x_test = x_test[:1000]

N_VISIBLE = 784
N_HIDDEN = 64
LEARNING_RATE = 0.01
BATCH_SIZE = 64
EPOCHS = 10

# 2. Initialize RBM parameters
W = np.random.normal(0, 0.01, (N_VISIBLE, N_HIDDEN)).astype(np.float32)
visible_bias = np.zeros(N_VISIBLE, dtype=np.float32)
hidden_bias = np.zeros(N_HIDDEN, dtype=np.float32)

# Numerically stable sigmoid
def sigmoid(x):
```

```
    x = np.clip(x, -30, 30)
    return 1.0 / (1.0 + np.exp(-x))

# 3. Energy function:
# E(v,h) = -a^T v - b^T h - v^T W h
def energy(v, h):
    return (
        -np.sum(v * visible_bias, axis=-1)
        -np.sum(h * hidden_bias, axis=-1)
        -np.sum((v @ W) * h, axis=-1)
    )

# Conditional activation probabilities
def hidden_probabilities(v):
    return sigmoid(v @ W + hidden_bias)

def visible_probabilities(h):
    return sigmoid(h @ W.T + visible_bias)

# Free energy for visible vectors:
# F(v) = -a^T v - sum_j log(1 + exp(b_j + (vW)_j))
def free_energy(v):
    hidden_input = v @ W + hidden_bias
    return (
        -np.sum(v * visible_bias, axis=1)
        -np.sum(np.logaddexp(0, hidden_input), axis=1)
    )

# 4. Train with one-step contrastive divergence (CD-1)
history_free_energy = []
history_reconstruction_error = []

n_batches = int(np.ceil(len(x_train) / BATCH_SIZE))

for epoch in range(EPOCHS):
    order = np.random.permutation(len(x_train))
    epoch_losses = []
    epoch_free_energy = []

    for start in range(0, len(x_train), BATCH_SIZE):
        batch_indices = order[start:start + BATCH_SIZE]
        v0 = x_train[batch_indices]
        batch_n = len(v0)
```

```
        # Positive phase
        ph0 = hidden_probabilities(v0)
        h0 = (np.random.random(ph0.shape) < ph0).astype(np.float32)

        # One Gibbs step: hidden -> visible -> hidden
        pv1 = visible_probabilities(h0)
        v1 = (np.random.random(pv1.shape) < pv1).astype(np.float32)

        ph1 = hidden_probabilities(v1)

        # Contrastive-divergence parameter updates
        positive_associations = v0.T @ ph0
        negative_associations = v1.T @ ph1

        W += LEARNING_RATE * (
            positive_associations - negative_associations
        ) / batch_n

        visible_bias += LEARNING_RATE * np.mean(v0 - v1, axis=0)
        hidden_bias += LEARNING_RATE * np.mean(ph0 - ph1, axis=0)

        # Batch reconstruction error and average free energy
        epoch_losses.append(np.mean((v0 - pv1) ** 2))
        epoch_free_energy.append(np.mean(free_energy(v0)))

    history_reconstruction_error.append(np.mean(epoch_losses))
    history_free_energy.append(np.mean(epoch_free_energy))

    print(
        f"Epoch {epoch + 1:2d}/{EPOCHS} | "
        f"Free energy: {history_free_energy[-1]:.4f} | "
        f"Reconstruction MSE: {history_reconstruction_error[-1]:.4f}"
    )

# 5. Plot free energy and reconstruction error
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(range(1, EPOCHS + 1), history_free_energy, marker="o")
axes[0].set_title("Average Free Energy")
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("Free Energy")
axes[0].grid(True)

axes[1].plot(
```

```
    range(1, EPOCHS + 1),
    history_reconstruction_error,
    marker="o"
)
axes[1].set_title("Reconstruction Error")
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Mean Squared Error")
axes[1].grid(True)

plt.tight_layout()
plt.show()

# 6. Reconstruct test images
n_show = 10
original = x_test[:n_show]

# Use hidden activation probabilities for a deterministic reconstruction
hidden_probs = hidden_probabilities(original)
reconstructed_probs = visible_probabilities(hidden_probs)

# Binarized versions for display
reconstructed_binary = (
    reconstructed_probs >= 0.5
).astype(np.float32)

# 7. Show original and reconstructed images side by side
fig, axes = plt.subplots(2, n_show, figsize=(15, 4))

for i in range(n_show):
    axes[0, i].imshow(original[i].reshape(28, 28), cmap="gray")
    axes[0, i].set_title("Original")
    axes[0, i].axis("off")

    axes[1, i].imshow(
        reconstructed_binary[i].reshape(28, 28), cmap="gray"
    )
    axes[1, i].set_title("Reconstructed")
    axes[1, i].axis("off")

plt.suptitle("RBM Reconstruction Results")
plt.tight_layout()
plt.show()

# 8. Report test reconstruction error
```

```
test_mse = np.mean((original - reconstructed_probs) ** 2)
print(f"Test reconstruction MSE on {n_show} images: {test_mse:.4f}")

# Example: compute energy for the first test image and a sampled hidden state
v_example = original[:1]
p_h = hidden_probabilities(v_example)
h_example = (np.random.random(p_h.shape) < p_h).astype(np.float32)

print("Energy of example (visible, hidden) state:",
      float(energy(v_example, h_example)[0]))
```

**Output:**

```
Epoch  1/10 | Free energy: -69.4933 | Reconstruction MSE: 0.1164
Epoch  2/10 | Free energy: -41.3155 | Reconstruction MSE: 0.0898
Epoch  3/10 | Free energy: -35.6772 | Reconstruction MSE: 0.0829
Epoch  4/10 | Free energy: -38.8951 | Reconstruction MSE: 0.0768
Epoch  5/10 | Free energy: -44.4048 | Reconstruction MSE: 0.0716
Epoch  6/10 | Free energy: -50.9948 | Reconstruction MSE: 0.0679
Epoch  7/10 | Free energy: -56.8231 | Reconstruction MSE: 0.0653
Epoch  8/10 | Free energy: -61.6999 | Reconstruction MSE: 0.0634
Epoch  9/10 | Free energy: -65.9485 | Reconstruction MSE: 0.0617
Epoch 10/10 | Free energy: -69.5773 | Reconstruction MSE: 0.0603
```

